Práctica 1: Introducción a la Programación de Smart Contracts en Solidity

Ejercicio 5: Contrato Inteligente Propio

Xulio Mougán Iglesias
Máster Universitario en Ciberseguridad (MUniCS)
Universidade de Vigo (UVigo) – Universidade da Coruña (UDC)

1. Análisis y definición del escenario

El escenario elegido consiste en la creación de una plataforma descentralizada para gestionar el alquiler y la fianza de una vivienda o habitación temporal de forma segura. Surge para solucionar un problema muy común en los alquileres: la desconfianza entre casero e inquilino, especialmente cuando el propietario no quiere devolver la fianza al marcharse o cuando una agencia cobra comisiones muy altas solo por hacer de intermediaria.

Para este caso es necesario usar una red blockchain pública como Ethereum, ya que permite que dos personas que no se conocen puedan realizar un acuerdo económico con total seguridad. En lugar de dejarle el dinero de la fianza directamente al casero o a una empresa privada, el importe queda bloqueado en el contrato inteligente. Así, nadie puede tocar los fondos a su antojo: el dinero solo se libera al casero si el alquiler se confirma correctamente, o se le devuelve de forma directa e íntegra al inquilino cuando termina el periodo pactado.

2. Diseño

2.1. Definición del caso de uso

Lógica de negocio: El sistema modela el ciclo de vida de un alquiler temporal entre dos partes independientes mediante las siguientes etapas secuenciales:

1. **Fase de oferta:** El propietario despliega el contrato en la red fijando los importes requeridos para la renta y la fianza. El contrato queda a la espera de un arrendatario en estado disponible.
2. **Fase de reserva y ocupación:** Un usuario interesado formaliza el alquiler enviando en una única transacción la suma exacta requerida. El contrato valida el pago, registra la dirección del usuario como inquilino y bloquea el estado para impedir ofertas concurrentes.
3. **Fase de liquidación:** Al concluir la estancia, el propietario valida la devolución de las llaves y el estado del inmueble mediante dos posibles vías:
   - *Liquidación favorable:* No hay daños, el alquiler se acredita al casero y la fianza se desbloquea íntegramente a favor del inquilino.
   - *Liquidación por daños:* Se documentan desperfectos, transfiriendo tanto el alquiler como la fianza al propietario para sufragar las reparaciones.

Automatización de transacciones: El contrato automatiza la custodia y los cobros mediante la máquina de estados de Ethereum:

- **Validación de depósito en tiempo de ejecución:** La función de entrada verifica de forma automática y atómica que el valor recibido coincida con la suma exigida, rechazando cualquier intento con saldo erróneo sin necesidad de verificación manual.
- **Custodia no permisionada:** Los fondos recibidos quedan bloqueados en el balance del contrato y ninguna de las dos partes puede transferirlos anticipadamente mientras el alquiler esté activo.
- **Liquidación desacoplada (Pull-Payment):** Una vez cerrado el alquiler, el contrato calcula internamente los balances asignados a cada dirección, permitiendo que cada usuario retire su capital de forma independiente y segura.

2.2. Contenido del contrato inteligente

Datos del contrato:

- `propietario` (`address payable`): Dirección de la cartera del dueño del inmueble que crea el contrato.
- `inquilino` (`address payable`): Dirección del usuario que alquila la vivienda.
- `precioAlquiler` (`uint256`): Coste del alquiler expresado en Wei.
- `fianza` (`uint256`): Importe del depósito de garantía expresado en Wei.
- `estado` (`enum Estado`): Variable que controla la fase actual del acuerdo (*Disponible*, *Alquilado*, *Finalizado*).
- `saldoPendiente` (`mapping(address => uint256)`): Registro del dinero desbloqueado que cada usuario tiene disponible para retirar de forma segura.

Funciones que operan sobre los datos:

- `alquilar()`: Función `payable` que registra al emisor como inquilino, recibe el Ether del alquiler y la fianza, y actualiza el estado a *Alquilado*.
- `finalizarSinIncidencias()`: Función ejecutada por el propietario al terminar la estancia que asigna el importe del alquiler al casero y desbloquea la fianza para el inquilino.
- `reclamarPorDanos()`: Función que permite al propietario retener la fianza en caso de desperfectos justificados al concluir el alquiler.
- `retirarFondos()`: Función que permite tanto al propietario como al inquilino transferir a sus respectivas carteras el saldo que les corresponde.

Reglas de operación:

- Ningún usuario puede alquilar el inmueble si su estado actual no es *Disponible*.
- La transacción de entrada se revierte automáticamente si el Ether adjunto (`msg.value`) no es exactamente igual a `precioAlquiler + fianza`.
- El propietario no puede retirar ni tocar el dinero depositado mientras el contrato permanezca en curso, garantizando que el capital esté bloqueado hasta el cierre.
- Las funciones de cierre solo pueden ser invocadas por el propietario mediante el modificador de acceso `soloPropietario`.

2.3. Usuarios del sistema

En el sistema interactúan dos tipos de usuarios con permisos diferenciados:

- **Propietario (Arrendador):** Es el creador del contrato. Se encarga de desplegarlo definiendo el precio del alquiler y la fianza, dar por finalizada la estancia al entregar las llaves y retirar el cobro del alquiler.
- **Inquilino (Arrendatario):** Es el cliente que consulta las condiciones públicas del contrato en la blockchain, realiza el pago inicial llamando a `alquilar()` y retira su fianza al finalizar el contrato.

Relación de casos de uso por actor:

- `[Propietario]` ⟶ (Desplegar contrato con precios) / (Finalizar alquiler) / (Retirar cobro)
- `[Inquilino]` ⟶ (Pagar alquiler + fianza) / (Retirar fianza devuelta)

3. Implementación

A continuación, se presenta el código fuente íntegro del contrato inteligente desarrollado en Solidity:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract AlquilerSeguro {
  address payable public immutable propietario;
  uint256 public immutable precioAlquiler;
  uint256 public immutable fianza;
  address payable public inquilino;

  enum Estado { Disponible, Alquilado, Finalizado }
  Estado public estado;

  mapping(address => uint256) public saldoPendiente;

  event PropiedadAlquilada(address indexed inquilino, uint256 totalDepositado);
  event AlquilerFinalizado(bool retencionPorDanos, uint256 fianzaDevuelta);
  event FondosRetirados(address indexed usuario, uint256 cantidad);

  modifier soloPropietario() {
    require(msg.sender == propietario, "Solo el propietario puede ejecutar esto");
    _;
  }

  modifier enEstado(Estado _estadoEsperado) {
    require(estado == _estadoEsperado, "Estado del contrato incompatible");
    _;
  }

  constructor(uint256 _precioEnEther, uint256 _fianzaEnEther) {
    require(_precioEnEther > 0, "El precio de alquiler debe ser mayor a 0");
    propietario = payable(msg.sender);
    precioAlquiler = _precioEnEther * 1 ether;
    fianza = _fianzaEnEther * 1 ether;
    estado = Estado.Disponible;
  }

```

```
37      function alquilar() external payable enEstado(Estado.Disponible) {
38          require(msg.value == (precioAlquiler + fianza), "Debe enviar el total
        exacto");
39          inquilino = payable(msg.sender);
40          estado = Estado.Alquilado;
41          emit PropiedadAlquilada(msg.sender, msg.value);
42      }
43
44      function finalizarSinIncidencias() external soloPropietario enEstado(Estado.
        Alquilado) {
45          estado = Estado.Finalizado;
46          saldoPendiente[propietario] += precioAlquiler;
47          saldoPendiente[inquilino] += fianza;
48          emit AlquilerFinalizado(false, fianza);
49      }
50
51      function reclamarPorDanos() external soloPropietario enEstado(Estado.
        Alquilado) {
52          estado = Estado.Finalizado;
53          saldoPendiente[propietario] += (precioAlquiler + fianza);
54          emit AlquilerFinalizado(true, 0);
55      }
56
57      function retirarFondos() external {
58          uint256 cantidad = saldoPendiente[msg.sender];
59          require(cantidad > 0, "No dispone de fondos pendientes para retirar");
60          saldoPendiente[msg.sender] = 0;
61          (bool exito, ) = payable(msg.sender).call{value: cantidad}("");
62          require(exito, "Fallo al transferir los fondos");
63          emit FondosRetirados(msg.sender, cantidad);
64      }
65
66      function obtenerBalanceContrato() external view returns (uint256) {
67          return address(this).balance;
68      }
69  }
```

4. Pruebas

Se ha realizado una batería de pruebas en el entorno de simulación local Remix VM para verificar el correcto funcionamiento del contrato inteligente a lo largo de todo su ciclo de vida.

4.1. Despliegue del contrato

El creador (propietario) inicializa el contrato inteligente configurando un precio de alquiler de 2 ETH y una fianza de 1 ETH mediante los parámetros pasados al constructor. Una vez confirmada la transacción, el contrato queda desplegado en la red con su dirección propia y sus funciones disponibles.

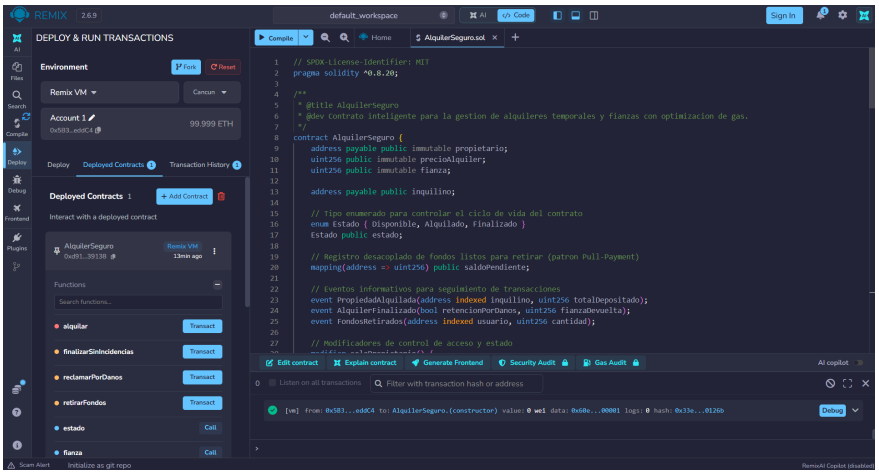


Figura 1: Despliegue inicial del contrato AlquilerSeguro en Remix IDE.

4.2. Reserva y depósito del inquilino

Una segunda cuenta que actúa como inquilino ejecuta la función `alquilar()` adjuntando un valor exacto de 3 ETH (2 ETH de renta más 1 ETH de fianza). La transacción se confirma con éxito en la consola, el saldo del inquilino se descuenta y el contrato pasa al estado *Alquilado*, reteniendo los fondos en su balance.

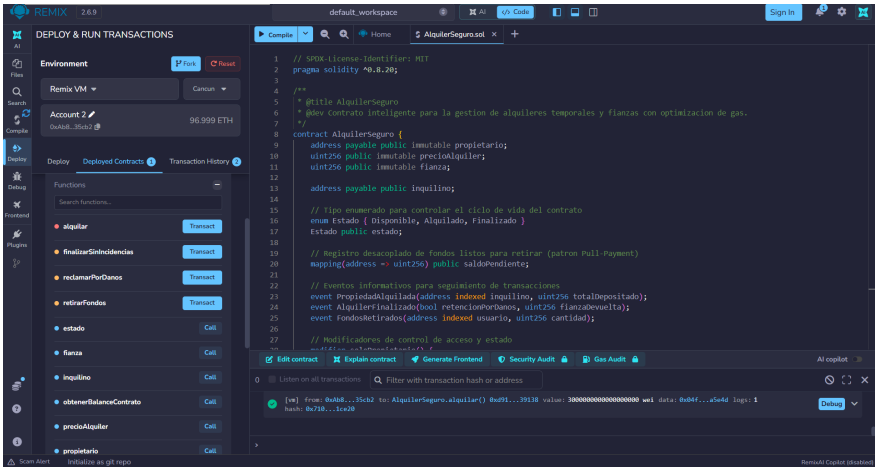


Figura 2: Pago de 3 ETH y formalización de la reserva por parte del inquilino.

4.3. Intento no autorizado de finalizar la estancia

Para comprobar las restricciones de seguridad y el control de acceso, la cuenta del inquilino intenta llamar a la función `finalizarSinIncidencias()`. La máquina virtual de Ethereum revierte la transacción de inmediato mostrando el mensaje de error: *"Solo el propietario puede ejecutar esto"*, impidiendo que un tercero no autorizado modifique el estado.

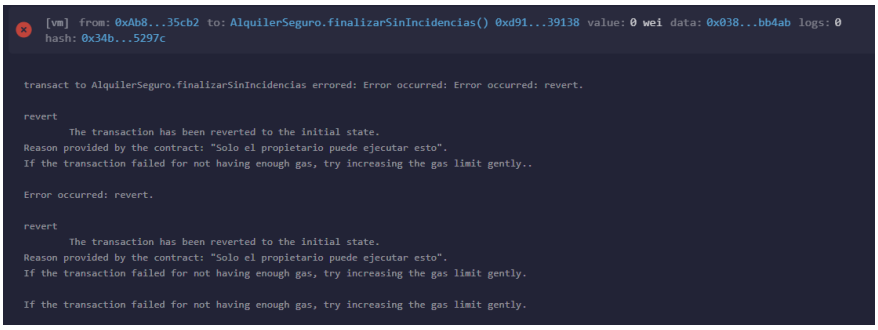


Figura 3: Reversión de la transacción ante una llamada sin permisos de propietario.

4.4. Finalización y retirada de fondos

Finalmente, el propietario legítimo ejecuta la función `finalizarSinIncidencias()`, cerrando el acuerdo y calculando los saldos pendientes en el registro interno. Acto seguido, el propietario invoca `retirarFondos()` para ingresar la renta de 2 ETH en su cartera, incrementando su balance de forma segura.

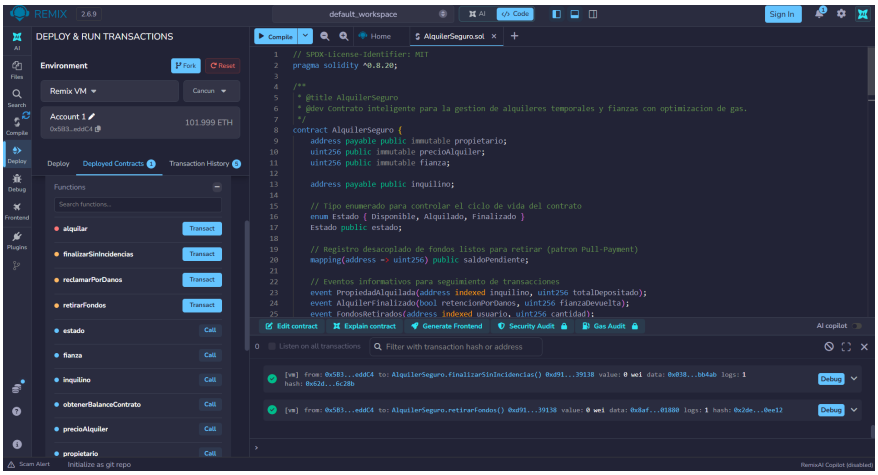


Figura 4: Finalización del alquiler y cobro de los fondos por el propietario.